23. STREAMS AND FILES

A stream is an object that provides an interface to a physical or logical device. The stream object contains local data and methods that operate on the stream object. Medley's general-purpose I/O functions take a stream as one of their arguments. Not every device is capable of implementing every I/O operation, while some devices offer special functions for that device alone. Such restrictions and extensions are noted in the documentation of each device. The majority of the streams used in Medley fall into two categories: file streams and image streams.

A file is a sequence of data stored on some device that allows the data to be retrieved at a later time. Files are identified by a name specifying their storage devices. Input or output to a file is performed through a stream to the file, using OPENSTREAM (below). In addition, there are functions that manipulate the files themselves, rather than their data content.

An image stream is an output stream to a display device, such as the display screen or a printer. In addition to the standard output operations, an image stream implements a variety of graphics operations, such as drawing lines and displaying characters in multiple fonts. Unlike a file, the "content" of an image stream cannot be retrieved. Image streams are described in Chapter 26.

This chapter describes operations specific to file devices: how to name files, how to open streams to files, and how to manipulate files on their devices.

Opening and Closing File Streams

To perform input from or output to a file, you must create a stream to the file, using OPENSTREAM:

(**OPENSTREAM** *FILE ACCESS RECOG PARAMETERS* —) [Function]

Opens and returns a stream for the file specified by *FILE*, a file name. *FILE* can be either a string or a symbol. The syntax and manipulation of file names is described at length in the FILENAMES section below. Incomplete file names are interpreted with respect to the connected directory (below).

RECOG specifies the recognition mode of *FILE* (below). If *RECOG* = NIL, it defaults according to the value of *ACCESS*.

ACCESS specifies the "access rights" to be used when opening the file. Possible values are:

INPUT — Only input operations are permitted on the already existing file. Starts reading at the beginning of the file. *RECOG* defaults to OLD.

OUTPUT — Only output operations are permitted on the initially empty file. Starts writing at the beginning of the file. While the file is open, other users or processes are unable to open the file for either input or output. *RECOG* defaults to NEW.

BOTH — Both input and output operations are permitted on the file. Starts reading or writing at the beginning of the file. *RECOG* defaults to OLD/NEW. *ACCESS* = BOTH implies random access (Chapter 25), and may not be possible for files on some devices.

APPEND — Only sequential output operations are permitted on the file. Starts

writing at the end of the file. *RECOG* defaults to OLD/NEW. *ACCESS* = APPEND may not be allowed for files on some devices.

Note: *ACCESS* = OUTPUT implies that you intend to write a new or different file, even if a version number was specified and the corresponding file already exists. Any previous contents of the file are discarded, and the file is empty immediately after the OPENSTREAM. If you want to write on an already existing file while preserving the old contents, the file must be opened for access BOTH or APPEND.

PARAMETERS is a list of pairs (*ATTRIB VALUE*), where *ATTRIB* is a file attribute (see SETFILEINFO below). A non-list *ATTRIB* in *PARAMETERS* is treated as the pair (*ATTRIB* T). Generally speaking, attributes that belong to the permanent file (e.g., TYPE) can only be set when creating a new file, while attributes that belong only to a particular opening of a file (e.g., ENDOFSTREAMOP) can be set on any call to OPENSTREAM. Not all devices honor all attributes; those not recognized by a particular device are simply ignored.

In addition to the attributes permitted by SETFILEINFO, the following attributes are accepted by OPENSTREAM as values of *ATTRIB* in its *PARAMETERS* argument:

DON'T.CHANGE.DATE — If *VALUE* is non-NIL, the file's creation date is not changed when the file is opened. This option is meaningful only for old files opened for BOTH access. You should use this only for specialized applications where the caller does not want the file system to believe the file's content has been changed.

SEQUENTIAL — If *VALUE* is non-NIL, this opening of the file need support only sequential access; i.e., the caller intends never to use SETFILEPTR. For some devices, sequential access to files is much more efficient than random access. Note that the device may choose to ignore this attribute and still open the file in a manner that permits random access. Also note that this attribute does not make sense with *ACCESS* = BOTH.

If *FILE* is not recognized by the file system, OPENSTREAM causes the error FILE NOT FOUND. Ordinarily, this error is intercepted via an entry on ERRORTYPELST (Chapter 24), which causes SPELLFILE (see the Searching File Directories below) to be called. SPELLFILE searches alternate directories and possibly attempts spelling correction on the file name. Only if SPELLFILE is unsuccessful will the FILE NOT FOUND error actually occur.

If *FILE* exists but cannot be opened, OPENSTREAM causes one of several other errors: FILE WON'T OPEN if the file is already opened for conflicting access by someone else; PROTECTION VIOLATION if the file is protected against the operation; FILE SYSTEM RESOURCES EXCEEDED if there is no more room in the file system.

(**CLOSEF** *FILE*) [Function]

Closes *FILE* and returns its full file name. Generates an error, FILE NOT OPEN, if *FILE* does not designate an open stream. After closing a stream, no further input/output operations are permitted on it.

If *FILE* is NIL, it is defaulted to the primary input stream if that is not the terminal

stream, or else the primary output stream if that is not the terminal stream. If both primary input and output streams are the terminal input/output streams, CLOSEF returns NIL. If CLOSEF closes either the primary input stream or the primary output stream (either explicitly or in the *FILE* = NIL case), it resets the primary stream for that direction to be the corresponding terminal stream. See Chapter 25 for information on the primary input/output streams.

WHENCLOSE (below) allows you to "advise" CLOSEF to perform various operations when a file is closed.

Because of buffering, the contents of a file open for output are not guaranteed to be written to the actual physical file device until CLOSEF is called. Buffered data can be forced out to a file without closing the file by using the function FORCEOUTPUT (Chapter 25).

Some network file devices perform their transactions in the background. As a result, it is possible for a file to be closed by CLOSEF and yet not be "fully" closed for a small time period afterward. During this time the file appears to be busy and cannot be opened for conflicting access by others.

(**CLOSEF?** *FILE*) [Function]

Closes *FILE* if it is open, returning the value of CLOSEF; otherwise does nothing and returns NIL.

In the present implementation of Medley, all open streams to files are kept in a registry of "open files". This registry does not include nameless streams, such as string streams (below), display streams (Chapter 28), and the terminal input and output streams; nor streams explicitly hidden from you, such as dribble streams (Chapter 30). This registry may not persist in future implementations of Medley, but at the present time it is accessible by the following two functions:

(**OPENP** *FILE ACCESS*) [Function]

ACCESS is an access mode for a stream opening (see OPENSTREAM), or NIL for any access.

If *FILE* is a stream, returns its full name if it is open for the specified access, otherwise NIL.

If *FILE* is a file name (a symbol), *FILE* is processed according to the rules of file recognition (below). If a stream open to a file by that name is registered and open for the specified access, then the file's full name is returned. If the file name is not recognized, or no stream is open to the file with the specified access, NIL is returned.

If *FILE* is NIL, returns a list of the full names of all registered streams that are open for the specified access.

(**CLOSEALL** *ALLFLG*) [Function]

Closes all streams in the value of (OPENP). Returns a list of the files closed.

WHENCLOSE (below) allows certain files to be "protected" from CLOSEALL. If *ALLFLG* is T, all files, including those protected by WHENCLOSE, are closed.

File Names

A file name in Medley is a string or symbol whose characters specify a "path" to the actual file: on what host or device the file resides, in which directory, and so forth. Because Medley supports a variety of non-local file devices, parts of the path could be device-dependent. However, it is desirable for programs to be able to manipulate file names in a device-independent manner. To this end, Medley specifies a uniform file name syntax over all devices; the functions that perform the actual file manipulation for a particular device are responsible for any translation to that device's naming conventions.

A file name is composed of a collection of *fields*, some of which have specific meanings. The functions described below refer to each field by a *field name*, a literal atom from among the following: `HOST`, `DEVICE`, `DIRECTORY`, `NAME`, `EXTENSION`, and `VERSION`. The standard syntax for a file name is `{HOST}DEVICE:<DIRECTORY>NAME.EXTENSION;VERSION`. Some host's file systems do not use all of those fields in their file names.

`HOST` Specifies the host whose file system contains the file. In the case of local file devices, the "host" is the name of the device, e.g., `DSK` or `FLOPPY`.

`DEVICE` Specifies, for those hosts that divide their file system's name space among mutiple physical devices, the device or logical structure on which the file resides. This should not be confused with Medley's abstract "file device", which denotes either a host or a local physical device and is specified by the `HOST` field.

`DIRECTORY` Specifies the "directory" containing the file. A directory usually is a grouping of a possibly large set of loosely related files, e.g., the personal files of a particular user, or the files belonging to some project. The `DIRECTORY` field usually consists of a principal directory and zero or more subdirectories that together describe a path through a file system's hierarchy. Each subdirectory name is set off from the previous directory or subdirectory by the character ">"; e.g., "`LISP>LIBRARY>NEW`".

`NAME` This field carries no specific meaning, but generally names a set of files thought of as being different renditions of the "same" abstract file.

`EXTENSION` This field also carries no specific meaning, but generally distinguishes the form of files having the same name. Most files systems have some "conventional" extensions that denote something about the content of the file. For example, in Medley, the extension `DCOM`, `LCOM` or `DFASL` denotes files containing compiled function definitions.

`VERSION` A number used to distinguish the versions or "generations" of the files having a common name and extension. The version number is incremented each time a new file by the same name is created.

Most functions that take as input "a directory" accept either a directory name (the contents of the `DIRECTORY` field of a file name) or a "full" directory specification—a file name fragment consisting of only the fields `HOST`, `DEVICE`, and `DIRECTORY`. In particular, the "connected directory" (see below) consists, in general, of all three fields.

For convenience in dealing with certain operating systems, Medley also recognizes [] and () as host delimiters (synonymous with {}), and / as a directory delimiter (synonymous with **<** at the beginning of a directory specification and **>** to terminate directory or subdirectory specification). For example, a file on a Unix file server `UNX` with the name `/usr/foo/bar/stuff.tedit`, whose `DIRECTORY` field is thus `usr/foo/bar`, could be specified as `{UNX}/usr/foo/bar/stuff.tedit`, or `(UNX)<usr/foo/bar>stuff.tedit`, or several other variations. Note that when using `[]` or `()` as host delimiters, they usually must be escaped with the reader's escape character if the file name is expressed as a symbol rather than a string.

Different hosts have different requirements for vaild characters in file names. In Medley, all characters are valid. However, in order to be able to parse a file name into its component fields, it is necessary that those characters that are conventionally used as file name delimiters be quoted when they appear inside of fields where there could be ambiguity. The file name quoting character is " ' " (single quote). Thus, the following characters must be quoted when not used as delimeters: **>**, :, ;, /, and ' itself. The character . (period) need only be quoted if it is to be considered a part of the `EXTENSION` field. The characters },], and) need only be quoted in a file name when the host field of the name is introduced by {, [, and (, respectively. The characters {, [, (, and **<** need only be quoted if they appear as the first character of a file name fragment, where they would otherwise be assumed to introduce the `HOST` or `DIRECTORY` fields.

The following functions are the standard way to manipulate file names in Medley. Their operation is purely syntactic—they perform no file system operations themselves.

(**UNPACKFILENAME.STRING** *FILENAME*) [Function]

Parses *FILENAME*, returning a list in property list format of alternating field names and field contents. The field contents are returned as strings. If it is a stream, its full name is used.

Only those fields actually present in *FILENAME* are returned. A field is considered present if its delimiting punctuation is present, even if the field itself is empty. Empty fields are denoted by "" (the empty string).

Examples:

```
(UNPACKFILENAME.STRING "FOO.BAR") =>
   (NAME "FOO" EXTENSION "BAR")

(UNPACKFILENAME.STRING "FOO.;2") =>
   (NAME "FOO" EXTENSION "" VERSION "2")

(UNPACKFILENAME.STRING "FOO;") =>
   (NAME "FOO" VERSION "")

(UNPACKFILENAME.STRING
   "{ERIS}<LISP>CURRENT>IMTRAN.DCOM;21")
 => (HOST "ERIS" DIRECTORY "LISP>CURRENT"
      NAME "IMTRAN" EXTENSION "DCOM"
      VERSION "21")
```

(**UNPACKFILENAME** *FILE*) [Function]

Old version of `UNPACKFILENAME.STRING` that returns the field values as atoms, rather than as strings. `UNPACKFILENAME.STRING` is now considered the "correct" way of unpacking file names, because it does not lose information when the contents of a field are numeric. For example,

```
(UNPACKFILENAME 'STUFF.TXT) =>
   (NAME STUFF EXTENSION TXT)
```

but

```
(UNPACKFILENAME 'STUFF.029) =>
   (NAME STUFF EXTENSION 29)
```

Explicitly omitted fields are denoted by the atom NIL, rather than the empty string.

Note: Both UNPACKFILENAME and UNPACKFILENAME.STRING leave the trailing colon on the device field, so that the Tenex device NIL: can be distinguished from the absence of a device. Although UNPACKFILENAME.STRING is capable of making the distinction, it retains this behavior for backward compatibility. Thus,

```
(UNPACKFILENAME.STRING '{TOAST}DSK:FOO) =>
   (HOST "TOAST" DEVICE "DSK:" NAME "FOO")
```

(**FILENAMEFIELD** *FILENAME FIELDNAME*) [Function]

Returns, as an atom, the contents of the *FIELDNAME* field of *FILENAME*. If *FILENAME* is a stream, its full name is used.

(**PACKFILENAME.STRING** $FIELD_1$ $CONTENTS_1$ ... $FIELD_N$ $CONTENTS_N$) [NoSpread Function]

Takes a sequence of alternating field names and field contents (atoms or strings), and returns the corresponding file name, as a string.

If PACKFILENAME.STRING is given a single argument, it is interpreted as a list of alternating field names and field contents. Thus PACKFILENAME.STRING and UNPACKFILENAME.STRING operate as inverses.

If the same field name is given twice, the *first* occurrence is used.

The contents of the field name DIRECTORY may be either a directory name or a full directory specification as described above.

PACKFILENAME.STRING also accepts the "field name" BODY to mean that its contents should itself be unpacked and spliced into the argument list at that point. This feature, in conjunction with the rule that fields early in the argument list override later duplicates, is useful for altering existing file names. For example, to provide a default field, place BODY first in the argument list, then the default fields. To override a field, place the new fields first and BODY last.

If the value of the BODY field is a stream, its full name is used.

Examples:

```
(PACKFILENAME.STRING 'DIRECTORY "LISP"
   'NAME "NET")
      =>  "<LISP>NET"

(PACKFILENAME.STRING 'NAME "NET"
   'DIRECTORY "{DSK}<LISPFILES>")
      => "{DSK}<LISPFILES>NET"
```

```
(PACKFILENAME.STRING 'DIRECTORY "{DSK}"
   'BODY "{TOAST}<FOO>BAR")
      => "{DSK}BAR"

(PACKFILENAME.STRING 'DIRECTORY "FRED"
   'BODY "{TOAST}<FOO>BAR")
      => "{TOAST}<FRED>BAR"

(PACKFILENAME.STRING 'BODY "{TOAST}<FOO>BAR"
   'DIRECTORY "FRED")
      => "{TOAST}<FOO>BAR"

(PACKFILENAME.STRING 'VERSION NIL
   'BODY "{TOAST}<FOO>BAR.DCOM;2")
      => "{TOAST}<FOO>BAR.DCOM"

(PACKFILENAME.STRING 'BODY "{TOAST}<FOO>BAR.DCOM"
   'VERSION 1)
      => "{TOAST}<FOO>BAR.DCOM;1"

(PACKFILENAME.STRING 'BODY "{TOAST}<FOO>BAR.DCOM;"
   'VERSION 1)
      => "{TOAST}<FOO>BAR.DCOM;"

(PACKFILENAME.STRING 'BODY "BAR.;1"
   'EXTENSION "DCOM")
      => "BAR.;1"

(PACKFILENAME.STRING 'BODY "BAR;1"
   'EXTENSION "DCOM")
      => "BAR.DCOM;1"
```

In the last two examples, note that in one case the extension is explicitly present in the body (as indicated by the preceding period), while in the other there is no indication of an extension, so the default is used.

(**PACKFILENAME** *FIELD*$_1$ *CONTENTS*$_1$ ... *FIELD*$_N$ *CONTENTS*$_N$) [NoSpread Function]

The same as PACKFILENAME.STRING, except that it returns the file name as a symbol, instead of a string.

Incomplete File Names

In general, it is not necessary to pass a complete file name (one containing all the fields listed above) to functions that take a file name as an argument. Interlisp supplies suitable defaults for certain fields (below). Functions that return names of actual files, however, always return the full file name.

If the version field is omitted from a file name, Interlisp performs version recognition, as described below.

If the host, device and/or directory field are omitted from a file name, Interlisp uses the currently connected directory. You can change the currently connected directory by by calling CNDIR (below) or using the programmer's assistant command CONN.

Defaults are added to the partially specified name "left to right" until a host, device or directory field is encountered. Thus, if the connected directory is {TWENTY}PS:<FRED>, then

```
BAR.DCOM means
   {TWENTY}PS:<FRED>BAR.DCOM

<GRANOLA>BAR.DCOM means
```

```
    {TWENTY}PS:<GRANOLA>BAR.DCOM

MTA0:<GRANOLA>BAR.DCOM means
    {TWENTY}MTA0:<GRANOLA>BAR.DCOM

{THIRTY}<GRANOLA>BAR.DCOM means
    {THIRTY}<GRANOLA>BAR.DCOM
```

In addition, if the partially specified name contains a subdirectory, but no principal directory, then the subdirectory is appended to the connected directory. For example,

```
ISO>BAR.DCOM means
    {TWENTY}PS:<FRED>ISO>BAR.DCOM
```

Or, if the connected directory is the Unix directory `{UNX}/usr/fred/`, then `iso/bar.dcom` means `{UNX}/usr/fred/iso/bar.dcom`, but `/other/bar.dcom` means `{UNX}/other/bar.dcom`.

(**CNDIR** *HOST/DIR*) [Function]

Connects to the directory *HOST/DIR*, which can either be a directory name or a full directory specification including host and/or device. If the specification includes just a host, and the host supports directories, the directory is defaulted to the value of (USERNAME); if the host is omitted, connection is made to another directory on the same host as before. If *HOST/DIR* is NIL, connects to the value of LOGINHOST/DIR.

CNDIR returns the full name of the now-connected directory. Causes an error, `Non-existent directory`, if *HOST/DIR* is not a valid directory.

Note that CNDIR does not necessarily require or provide any directory access privileges. Access privileges are checked when a file is opened.

CONN *HOST/DIR* [Prog. Asst. Command]

Command form of CNDIR for use at the executive. Connects to *HOST/DIR*, or to the value of LOGINHOST/DIR if *HOST/DIR* is omitted. This command is undoable. —Undoing it causes the system to connect to the previously connected directory.

LOGINHOST/DIR [Variable]

CONN with no argument connects to the value of the variable LOGINHOST/DIR, initially {DSK}, but usually reset in your greeting file (Chapter 12).

(**DIRECTORYNAME** *DIRNAME STRPTR*) [Function]

If *DIRNAME* is T, returns the full specification of the currently connected directory. If *DIRNAME* is NIL, returns the value of LOGINHOST/DIR. For any other value of *DIRNAME*, returns a full directory specification if *DIRNAME* designates an existing directory (satisfies DIRECTORYNAMEP), otherwise NIL.

If *STRPTR* is T, the value is returned as an atom, otherwise it is returned as a string.

(**DIRECTORYNAMEP** *DIRNAME HOSTNAME*) [Function]

Returns T if *DIRNAME* is a valid directory on host *HOSTNAME*, or on the host of the currently connected directory if *HOSTNAME* is NIL. *DIRNAME* may be either a directory

name or a full directory specification containing host and/or device.

If *DIRNAME* includes subdirectories, this function may or may not pass judgment on their validity. Some hosts support "true" subdirectories, distinct entities manipulable by the file system, while others only provide them as a syntactic convenience.

(**HOSTNAMEP** *NAME*) [Function]

Returns T if *NAME* is recognized as a valid host or file device name at the moment HOSTNAMEP is called.

Version Recognition

Most of the file devices in Interlisp support file version numbers. That is, you can have several files of the exact same name, differing only in their VERSION field, which is incremented for each new "version" of the file that is created. When the filesystem encounters a file name without a version number, it must figure out which version was intended. This process is known as *version recognition*.

When OPENSTREAM opens a file for input and no version number is given, the highest existing version number is used. Similarly, when a file is opened for output and no version number is given, a new file is created with a version number one higher than the highest one currently in use with that file name. You can change he version number defaulting for OPENSTREAM by specifying a different value for its *RECOG* argument (see FULLNAME below).

Other functions that accept file names as arguments generally perform default version recognition, which is newest version for existing files, or a new version if using the file name to create a new file. The one exception is DELFILE, which uses the oldest existing version of the file.

The functions below can be used to perform version recognition without actually calling OPENSTREAM to open the file. Note that these functions only tell the truth at the moment they are called, and thus cannot be used to anticipate the name of the file opened by a comparable OPENSTREAM. They are best used as helpful hints.

(**FULLNAME** *X RECOG*) [Function]

If *X* is an open stream, simply returns the full file name of the stream. Otherwise, if *X* is a file name given as a string or symbol, performs version recognition, as follows:

If *X* is recognized in the recognition mode specified by *RECOG* as an abbreviation for some file, returns the file's full name, otherwise NIL. *RECOG* is one of the following:

OLD Chooses the newest existing version of the file. Returns NIL if no file named *X* exists.

OLDEST Chooses the oldest existing version of the file. Returns NIL if no file named *X* exists.

NEW Chooses a new version of the file. If versions of *X* already exist, then chooses a version number one higher than highest existing version; otherwise chooses version 1. For some file systems, FULLNAME returns NIL if you do not have the access rights necessary to create a new file named *X*.

OLD/NEW Tries OLD, then NEW. Choose the newest existing version of the file, if any; otherwise chooses version 1. This usually only makes sense if you intend to open *X* for access BOTH.

RECOG = NIL defaults to OLD. For all other values of *RECOG*, generates an error ILLEGAL ARG.

If *X* already contains a version number, the *RECOG* argument will never change it. In particular, *RECOG* = NEW does not require that the file actually be new. For example, (FULLNAME 'FOO.;2 'NEW) may return {ERIS}<LISP>FOO.;2 if that file already exists, even though (FULLNAME 'FOO 'NEW) would default the version to a new number, perhaps returning {ERIS}<LISP>FOO.;5.

(**INFILEP** *FILE*) [Function]

Equivalent to (FULLNAME *FILE*'OLD). Returns the full file name of the newest version of *FILE* if *FILE* is the name of an existing file that can be opened for input, NIL otherwise.

(**OUTFILEP** *FILE*) [Function]

Equivalent to (FULLNAME *FILE*'NEW).

Note that INFILEP, OUTFILEP and FULLNAME do not open any files; they are pure predicates. They are also only hints, as they do not imply that the caller has access rights to the file. For example, INFILEP might return non-NIL, but OPENSTREAM might fail for the same file because you don't have read access to it, or the file is open for output by another user. Similarly, OUTFILEP could return non-NIL, but OPENSTREAM could fail with a FILE SYSTEM RESOURCES EXCEEDED error.

Note also that in a shared file system, such as a remote file server, intervening file operations by another user could contradict the information returned by recognition. For example, a file that was INFILEP might be deleted, or between an OUTFILEP and the subsequent OPENSTREAM, another user might create a new version or delete the highest version, causing OPENSTREAM to open a different version of the file than the one returned by OUTFILEP. In addition, some file servers do not support recognition of files in output context. Thus, the "truth" about a file can only be obtained by actually opening the file; creators of files should rely on the name of the stream opened by OPENSTREAM, not the value returned from these recognition functions. In particular, programmers are discouraged from using OUTFILEP or (FULLNAME *NAME*'NEW).

Using File Names Instead of Streams

In earlier implementations of Interlisp, from the days of Interlisp-10 onward, the "handle" used to refer to an open file was not a stream, but rather the file's full name, represented as a symbol. When the file name was passed to any I/O function, it was mapped to a stream by looking it up in a list of open files. This scheme was sometimes convenient for typing in file commands at the executive, but was poor for serious programming in two ways. First, mapping from file name to stream on every input/output operation is inefficient. Second, and more importantly, using the file name as the handle on an open stream means that it is not possible to have more than one stream open on a given file at once.

As of this writing, Medley is in a transition period, where it still supports the use of symbol file names

as synonymous with open streams, but this use is not recommended. The remainder of this section discusses this usage of file names for the benefit of those reading older programs and wishing to convert them to work properly when this compatibility feature is removed.

File Name Efficiency Considerations

It is possible for a program to be seriously inefficient using a file name as a stream if the program is not using the name returned by OPENFILE (below). Any time that an input/output function is called with a file name other than the full file name, Interlisp must perform recognition on the partial file name to determine which open file is intended. Thus if repeated operations are to be performed, it is considerably more efficient to use the full file name returned from OPENFILE.

There is a more subtle problem with partial file names, in that recognition is performed on your entire directory, not just the open files. It is possible for a file name that previously denoted one file to suddenly denote a different file. For example, suppose a program performs (INFILE 'FOO), opening FOO.;1, and reads several expressions from FOO. Then you interrupt the program, create a FOO.;2 and resume the program (or a user at another workstation creates a FOO.;2). Now a call to READ giving it FOO as its *FILE* argument will generate a FILE NOT OPEN error, because FOO will be recognized as FOO.;2.

Obsolete File Opening Functions

The following functions are now obsolete, but are provided for backwards compatibility:

(**OPENFILE** *FILE ACCESS RECOG PARAMETERS*) [Function]

Opens *FILE* with access rights as specified by *ACCESS*, and recognition mode *RECOG*, and returns the full name of the resulting stream. Equivalent to (FULLNAME (OPENSTREAM *FILE ACCESS RECOG PARAMETERS*)).

(**INFILE** *FILE*) [Function]

Opens *FILE* for input, and sets it as the primary input stream. Equivalent to (INPUT (OPENSTREAM *FILE* 'INPUT 'OLD))

(**OUTFILE** *FILE*) [Function]

Opens *FILE* for output, and sets it as the primary output stream. Equivalent to (OUTPUT (OPENSTREAM *FILE* 'OUTPUT 'NEW)).

(**IOFILE** *FILE*) [Function]

Opens *FILE* for both input and output. Equivalent to (OPENFILE *FILE* 'BOTH 'OLD). Does not affect the primary input or output stream.

Converting Old Programs

At some point in the future, the Medley file system will change so that each call to OPENSTREAM returns a distinct stream, even if a stream is already open to the specified file. This change is required in order to deal with files in a multiprocessing environment.

This change will produce the following incompatibilities:

1. The functions OPENFILE, INPUT, and OUTPUT will return a stream, not a full file

name. To make this less confusing in interactive situations, streams will have a print format that reveals the underlying file's actual name.

2. Passing anything other than the object returned from OPENFILE to I/O operations will cause problems. Passing the file's name will be significantly slower than passing the stream (even when passing the "full" file name), and in the case where there is more than one stream open on the file it might even act on the wrong one.

3. OPENP will return NIL when passed the name of a file rather than the value of OPENFILE or OPENSTREAM.

You should consider the following advice when writing new programs and editing existing programs, so your programs will behave properly when the change occurs:

Because of the efficiency and ambiguity considerations described earlier, users have long been encouraged to use only full file names as *FILE* arguments to I/O operations. The "proper" way to have done this was to bind a variable to the value returned from OPENFILE and pass that variable to all I/O operations; such code will continue to work. A less proper way to obtain the full file name, but one which has to date not incurred any obvious penalty, is that which binds a variable to the result of an INFILEP and passes that to OPENFILE and all I/O operations. This has worked because INFILEP and OPENFILE both return a full file name, an invalid assumption in this future world. Such code should be changed to pass around the value of the OPENFILE, not the INFILEP.

Code that calls OPENP to test whether a possibly incomplete file name is already open should be recoded to pass to OPENP only the value returned from OPENFILE or OPENSTREAM.

Code that uses ordinary string functions to manipulate file names, and in particular the value returned from OPENFILE, should be changed to use the the functions UNPACKFILENAME.STRING and PACKFILENAME.STRING. Those functions work both on file names (strings) and streams (coercing the stream to the name of its file).

Code that tests the value of OUTPUT for equality to some known file name or T should be examined carefully and, if possible, recoded.

To see more directly the effects of passing around streams instead of file names, replace your calls to OPENFILE with calls to OPENSTREAM. OPENSTREAM is called in exactly the same way, but returns a STREAM. Streams can be passed to READ, PRINT, CLOSEF, etc just as the file's full name can be currently, but using them is more efficient. The function FULLNAME, when applied to a stream, returns its full file name.

Using Files with Processes

Because Medley does not yet support multiple streams per file, problems can arise if different processes attempt to access the same file. You have to be careful not to have two processes manipulating the same file at the same time, since the two processes will be sharing a single input stream and file pointer. For example, you can't have one process TCOMPL a file while another process is running LISTFILES on it.

File Attributes

Any file has a number of "file attributes", such as the read date, protection, and bytesize. The exact attributes that a file can have is dependent on the file device. The functions GETFILEINFO and

SETFILEINFO allow you to access file attributes:

(**GETFILEINFO** *FILE ATTRIB*) [Function]

Returns the current setting of the *ATTRIB* attribute of *FILE*.

(**SETFILEINFO** *FILE ATTRIB VALUE*) [Function]

Sets the attribute *ATTRIB* of *FILE* to be *VALUE*. SETFILEINFO returns T if it is able to change the attribute *ATTRIB*, and NIL if unsuccessful, either because the file device does not recognize *ATTRIB* or because the file device does not permit the attribute to be modified.

The *FILE* argument to GETFILEINFO and SETFILEINFO can be an open stream (or an argument designating an open stream, see Chapter 25), or the name of a closed file. SETFILEINFO in general requires write access to the file.

The attributes recognized by GETFILEINFO and SETFILEINFO fall into two categories: *permanent* attributes, which are properties of the file, and *temporary* attributes, which are properties only of an open stream to the file. The temporary attributes are only recognized when *FILE* designates an open stream; the permanent attributes are usually equally accessible for open and closed files. However, some devices are willing to change the value of certain attributes of an open stream only when specified in the PARAMETERS argument to OPENSTREAM (see above), not on a later call to SETFILEINFO.

The following are permanent attributes of a file:

BYTESIZE The byte size of the file. Medley currently only supports byte size 8.

LENGTH The number of bytes in the file. Alternatively, the byte position of the end-of-file. Like (GETEOFPTR *FILE*), but *FILE* does not have to be open.

SIZE The size of *FILE* in pages.

CREATIONDATE The date and time, as a string, that the content of *FILE* was "created". The creation date changes whenever the content of the file is modified, but remains unchanged when a file is transported, unmodified, across file systems. Specifically, COPYFILE and RENAMEFILE (see below) preserve the file's creation date. Note that this is different from the concept of "creation date" used by some operating systems (e.g., Tops20).

WRITEDATE The date and time, as a string, that the content of *FILE* was last written to this particular file system. When a file is copied, its creation date does not change, but its write date becomes the time at which the copy is made.

READDATE The date and time, as a string, that *FILE* was last read, or NIL if it has never been read.

ICREATIONDATE
IWRITEDATE

IREADDATE The CREATIONDATE, WRITEDATE and READDATE, respectively, in integer form, as IDATE (Chapter 12) would return. This form is useful for comparing dates.

AUTHOR The name of the user who last wrote the file.

TYPE The "type" of the file, some indication of the nature of the file's content. The "types" of files allowed depends on the file device. Most devices recognize the symbol TEXT to mean that the file contains just characters, or BINARY to mean that the file contains arbitrary data.

Some devices support a wider range of file types that distinguish among the various sorts of files one might create whose content is "binary". All devices interpret any value of TYPE that they do not support to be BINARY. Thus, GETFILEINFO may return the more general value BINARY instead of the original type that was passed to SETFILEINFO or OPENSTREAM. Similarly, COPYFILE, while attempting to preserve the TYPE of the file it is copying, may turn, say, an INTERPRESS file into a mere BINARY file.

The way in which some file devices (e.g., Xerox file servers) support a wide range of file types is by representing the type as an integer, whose interpretation is known by the client. The variable FILING.TYPES is used to associate symbolic types with numbers for these devices. This list initially contains some of the well-known assignments of type name to number; you can add additional elements to handle any private file types. For example, suppose there existed an NS file type MAZEFILE with numeric value 5678. You could add the element (MAZEFILE 5678) to FILING.TYPES and then use MAZEFILE as a value for the TYPE attribute to SETFILEINFO or OPENSTREAM. Other devices are, of course, free to store TYPE attributes in whatever manner they wish, be it numeric or symbolic. FILING.TYPES is merely considered the official registry for Xerox file types.

For most file devices, the TYPE of a newly created file, if not specified in the *PARAMETERS* argument to OPENSTREAM, defaults to the value of DEFAULTFILETYPE, initially TEXT.

The following are currently recognized as temporary attributes of an open stream:

ACCESS The current access rights of the stream (see the beginning of this chapter). Can be one of INPUT, OUTPUT, BOTH, APPEND; or NIL if the stream is not open.

ENDOFSTREAMOP The action to be taken when a stream is at "end of file" and an attempt is made to take input from it. The value of this attribute is a function of one argument, the stream. The function can examine the stream and its calling context and take any action it wishes. If the function returns normally, its should return either T, meaning to try the input operation again, or the byte that BIN would have returned had there been more

bytes to read. Ordinarily, one should not let the ENDOFSTREAMOP function return unless one is only performing binary input from the file, since there is no way in general of knowing in what state the reader was at the time the end of file occurred, and hence how it will interpret a single byte returned to it.

The default ENDOFSTREAMOP is a system function that causes the error END OF FILE. The behavior of that error can be further modified for a particular stream by using the EOF option of WHENCLOSE (see below).

EOL The end-of-line convention for the stream. This can be CR, LF, or CRLF, indicating with what byte or sequence of bytes the "End Of Line" character is represented on the stream. On input, that sequence of bytes on the stream is read as (CHARCODE EOL) by READCCODE or the string reader. On output, (TERPRI) and (PRINTCCODE (CHARCODE EOL)) cause that sequence of bytes to be placed on the stream.

The end of line convention is usually not apparent to you. The file system is usually aware of the convention used by a particular remote operating system, and sets this attribute accordingly. If you believe a file actually is stored with a different convention than the default, it is possible to modify the default behavior by including the EOL attribute in the *PARAMETERS* argument to OPENSTREAM.

BUFFERS Value is the number of 512-byte buffers that the stream maintains at one time. This attribute is only used by certain random-access devices (currently, the local disk, floppy, and Leaf servers); all others ignore it.

Streams open to files generally maintain some portion of the file buffered in memory, so that each call to an I/O function does not require accessing the actual file on disk or a file server. For files being read or written sequentially, not much buffer space is needed, since once a byte is read or written, it will never need to be seen again. In the case of random access streams, buffering is more complicated, since a program may jump around in the file, using SETFILEPTR (Chapter 25). In this case, the more buffer space the stream has, the more likely it is that after a SETFILEPTR to a place in the file that has already been accessed, the stream still has that part of the file buffered and need not go out to the device again. This benefit must, of course, be traded off against the amount of memory consumed by the buffers.

NS servers implement the following additional attributes for GETFILEINFO (neither of these attributes are settable with SETFILEINFO):

READER The name of the user who last read the file.

PROTECTION A list specifying the access rights to the file. Each element of the list is of the form (name nametype . rights). Name is the name of a user or group or a name pattern. Rights is one or more of the symbols ALL READ WRITE DELETE CREATE or MODIFY. For servers running services 10.0 or later, nametype is the symbol "--". , In earlyer releases it is one of the symbols INDIVIDUAL or GROUP

Closing and Reopening Files

The function WHENCLOSE permits you to associate certain operations with open streams that govern how and when the stream will be closed. You can specify that certain functions will be executed before CLOSEF closes the stream and/or after CLOSEF closes the stream. You can make a particular stream be invisible to CLOSEALL, so that it will remain open across user invocations of CLOSEALL.

(**WHENCLOSE** *FILE* $PROP_1$ VAL_1 ... $PROP_N$ VAL_N) [NoSpread Function]

FILE must designate an open stream other than T (NIL defaults to the primary input stream, if other than T, or primary output stream if other than T). The remaining arguments specify properties to be associated with the full name of *FILE*. WHENCLOSE returns the full name of *FILE* as its value.

WHENCLOSE recognizes the following property names:

BEFORE *VAL* is a function that CLOSEF will apply to the stream just before it is closed. This might be used, for example, to copy information about the file from an in-core data structure to the file just before it is closed.

AFTER *VAL* is a function that CLOSEF will apply to the stream just after it is closed. This capability permits in-core data structures that know about the stream to be cleaned up when the stream is closed.

CLOSEALL *VAL* is either YES or NO and determines whether *FILE* will be closed by CLOSEALL (YES) or whether CLOSEALL will ignore it (NO). CLOSEALL uses CLOSEF, so that any AFTER functions will be executed if the stream is in fact closed. Files are initialized with CLOSEALL set to YES.

EOF *VAL* is a function that will be applied to the stream when an end-of-file error occurs, and the ERRORTYPELST entry for that error, if any, returns NIL. The function can examine the context of the error, and can decide whether to close the stream, RETFROM some function, or perform some other computation. If the function supplied returns normally (i.e., does not RETFROM some function), the normal error machinery will be invoked.

The default EOF behavior, unless overridden by this WHENCLOSE option, is to call the value of DEFAULTEOFCLOSE (below).

For some applications, the ENDOFSTREAMOP attribute (see above) is a more useful way to intercept the end-of-file error. The ENDOFSTREAMOP attribute comes into effect before the error machinery is ever activated.

Multiple AFTER and BEFORE functions may be associated with a file; they are executed in sequence with the most recently associated function executed first. The CLOSEALL and EOF values, however, will override earlier values, so only the last value specified will have an effect.

DEFAULTEOFCLOSE [Variable]

Value is the name of a function that is called by default when an end of file error occurs and no EOF option has been specified for the stream by WHENCLOSE. The initial value of DEFAULTEOFCLOSE is NILL, meaning take no special action (go ahead and cause the error). Setting it to CLOSEF would cause the stream to be closed before the rest of the error machinery is invoked.

I/O Operations to and from Strings

It is possible to treat a string as if it were the contents of a file by using the following function:

(**OPENSTRINGSTREAM** *STR ACCESS*) [Function]

Returns a stream that can be used to access the characters of the string *STR*. *ACCESS* may be either INPUT, OUTPUT, or BOTH; NIL defaults to INPUT. The stream returned may be used exactly like a file opened with the same access, except that output operations may not extend past the end of the original string. Also, string streams do not appear in the value of (OPENP).

For example, after performing

```
(SETQ STRM (OPENSTRINGSTREAM "THIS 2 (IS A LIST)"))
```

the following succession of reads could occur:

```
(READ STRM)   => THIS
   (RATOM STRM) => 2
   (READ STRM)  => (IS A LIST)
   (EOFP STRM)  => T
```

Compatibility Note: In Interlisp-10 it was possible to take input from a string simply by passing the string as the *FILE* argument to an input function. In order to maintain compatibility with this feature, Medley provides the same capability. This not terribly clean feature persists in the present implementation to give users time to convert old code. This means that strings are *not* equivalent to symbols when specifying a file name as a stream argument. In a future release, the old Interlisp-10 string-reading feature will be decommissioned, and OPENSTRINGSTREAM will be the only way to perform I/O on a string.

Temporary Files and the CORE Device

Many operating systems have a notion of "scratch file", a file typically used as temporary storage for data most naturally maintained in the form of a file, rather than some other data structure. A scratch file can be used as a normal file in most respects, but is automatically deleted from the file system after its useful life is up, e.g., when the job terminates, or you log out. In normal operation, you need never explicitly delete such files, since they are guaranteed to disappear soon.

A similar functionality is provided in Medley by core-resident files. Core-resident files are on the device CORE. The directory structure for this device and all files on it are represented completely within your virtual memory. These files are treated as ordinary files by all file operations; their only distinguishing feature is that all trace of them disappears when the Medley image is abandoned.

Core files are opened and closed by name the same as any other file, e.g., (OPENSTREAM

'{CORE}<FOO>FIE.DCOM 'OUTPUT). Directory names are completely optional, so files can also have names of the form {CORE}NAME.EXT. Core files can be enumerated by DIRECTORY (see below). While open, they are registered in (OPENP). They do consume virtual memory space, which is only reclaimed when the file is deleted. Some caution should thus be used when creating large CORE files. Since the virtual memory of an Medley workstation usually persists far longer than the typical process on a mainframe computer, it is still important to delete CORE files after they are no longer in use.

For many applications, the name of the scratch file is irrelevant, and there is no need for anyone to have access to the file independent of the program that created it. For such applications, NODIRCORE files are preferable. Files created on the device lisp NODIRCORE are core-resident files that have no name and are registered in no directory. These files "disappear", and the resources they consume are reclaimed, when all pointers to the file are dropped. Hence, such files need never be explicitly deleted or, for that matter, closed. The "name" of such a file is simply the stream object returned from (OPENSTREAM '{NODIRCORE} 'OUTPUT), and it is this stream object that must be passed to all input/output operations, including CLOSEF and any calls to OPENSTREAM to reopen the file.

(**COREDEVICE** *NAME NODIRFLG*) [Function]

Creates a new device for core-resident files and assigns *NAME* as its device name. Thus, after performing (COREDEVICE 'FOO), one can execute (OPENSTREAM '{FOO}BAR 'OUTPUT) to open a file on that device. Medley is initialized with the single core-resident device named CORE, but COREDEVICE may be used to create any number of logically distinct core devices.

If *NODIRFLG* is non-NIL, a core device that acts like {NODIRCORE} is created.

Compatibility note: In Interlisp-10, it was possible to create scratch files by using file names with suffixes ;S or ;T. In Medley, these suffixes in file names are simply ignored when output is directed to a particular host or device. However, the function PACKFILENAME.STRING is defined to default the device name to CORE if the file has the TEMPORARY attribute and no explicit host is provided.

NULL Device

The NULL device provides a source of content-free "files". (OPENSTREAM '{NULL} 'OUTPUT) creates a stream that discards all output directed at it. (OPENSTREAM '{NULL} 'INPUT) creates a stream that is perpetually at end-of-file (i.e., has no input).

Deleting, Copying, and Renaming Files

(**DELFILE** *FILE*) [Function]

Deletes *FILE* if possible. The file must be closed. Returns the full name of the file if deleted, else NIL. Recognition mode for *FILE* is OLDEST, i.e., if *FILE* does not have a version number specified, then DELFILE deletes the oldest version of the file.

(**COPYFILE** *FROMFILE TOFILE*) [Function]

Copies *FROMFILE* to a new file named *TOFILE*. The source and destination may be on any combination of hosts/devices. COPYFILE attempts to preserve the TYPE and CREATIONDATE where possible. If the original file's file type is unknown, COPYFILE

attempts to infer the type (file type is BINARY if any of its 8-bit bytes have their high bit on).

COPYFILE uses COPYCHARS (Chapter 25) if the source and destination hosts have different EOL conventions. Thus, it is possible for the source and destination files to be of different lengths.

(**RENAMEFILE** *OLDFILE NEWFILE*) [Function]

Renames *OLDFILE* to be *NEWFILE*. Causes an error, FILE NOT FOUND if *FILE* does not exist. Returns the full name of the new file, if successful, else NIL if the rename cannot be performed.

If *OLDFILE* and *NEWFILE* are on the same host/device, and the device implements a renaming primitive, RENAMEFILE can be very fast. However, if the device does not know how to rename files in place, or if *OLDFILE* and *NEWFILE* are on different devices, RENAMEFILE works by copying *OLDFILE* to *NEWFILE* and then deleting *OLDFILE*.

Searching File Directories

DIRECTORIES [Variable]

Global variable containing the list of directories searched (in order) by SPELLFILE and FINDFILE (below) when not given an explicit *DIRLST* argument. In this list, the atom NIL stands for the login directory (the value of LOGINHOST/DIR), and the atom T stands for the currently connected directory. Other elements should be *full* directory specifications, e.g., {TWENTY}PS:<LISPUSERS>, not merely LISPUSERS.

LISPUSERSDIRECTORIES [Variable]

Global variable containing a list of directories to search for "library" package files. Used by the FILES file package command (Chapter 17).

(**SPELLFILE** *FILE NOPRINTFLG NSFLG DIRLST*) [Function]

Searches for the file name *FILE*, possibly performing spelling correction (see Chapter 20). Returns the corrected file name, if any, otherwise NIL.

If *FILE* has a directory field, SPELLFILE attempts spelling correction against the files in that particular directory. Otherwise, SPELLFILE searches for the file on the directory list *DIRLST* before attempting any spelling correction.

If *NOPRINTFLG* is NIL, SPELLFILE asks you to confirm any spelling correction done, and prints out any files found, even if spelling correction is not done. If *NOPRINTFLG* = T, SPELLFILE does not do any printing, nor ask for approval.

If *NSFLG* = T (or NOSPELLFLG = T, see Chapter 20), no spelling correction is attempted, though searching through *DIRLST* still occurs.

DIRLST is the list of directories searched if *FILE* does not have a directory field. If *DIRLST* is NIL, the value of the variable DIRECTORIES is used.

Note: If *DIRLST* is NIL, and *FILE* is not found by searching the directories

on DIRECTORIES, but the root name of *FILE* has a FILEDATES property (Chapter 17) indicating that a file by that name has been loaded, then the directory indicated in the FILEDATES property is searched, too. This additional search is not done if *DIRLST* is non-NIL.

ERRORTYPELST (Chapter 14) initially contains the entry ((23 (SPELLFILE (CADR ERRORMESS) NIL NOFILESPELLFLG))), which causes SPELLFILE to be called in case of a FILE NOT FOUND error. If the variable NOFILESPELLFLG is T (its initial value), then spelling correction is not done on the file name, but DIRECTORIES is still searched. If SPELLFILE is successful, the operation will be reexecuted with the new (corrected) file name.

(**FINDFILE** *FILE NSFLG DIRLST*) [Function]

Uses SPELLFILE to search for a file named *FILE*. If it finds one, returns its full name, with no user interaction. Specifically, it calls (SPELLFILE *FILE* T *NSFLG DIRLST*), after first performing two simple checks: If *FILE* has an explicit directory, it checks to see if a file so named exists, and if so returns that file. If *DIRLST* is NIL, it looks for *FILE* on the connected directory before calling SPELLFILE.

Listing File Directories

The function DIRECTORY allows you to conveniently specify and/or program a variety of directory operations:

(**DIRECTORY** *FILES COMMANDS DEFAULTEXT DEFAULTVERS*) [Function]

Returns, lists, or performs arbitrary operations on all files specified by the "file group" *FILES*. A file group has the form of a regular file name, except that the character * can be used to match any number of characters, including zero, in the file name. For example, the file group A*B matches all file names beginning with the character A and ending with the character B. The file group *.DCOM matches all files with an extension of DCOM.

If *FILES* does not contain an explicit extension, it is defaulted to *DEFAULTEXT*; if *FILES* does not contain an explicit version, it is defaulted to *DEFAULTVERS*. *DEFAULTEXT* and *DEFAULTVERS* themselves default to *. If the period or semicolon preceding the omitted extension or version, respectively, is present, the field is explicitly empty and no default is used. All other unspecified fields default to *. Null version is interpreted as "highest". Thus *FILES* = * or *.* or *.*;* enumerates all files on the connected directory; *FILES* = *. or *.;* enumerates all versions of files with null extension; *FILES* = *.; enumerates the highest version of files with null extension; and *FILES* = *.*; enumerates the highest version of all files. If *FILES* is NIL, it defaults to *.*;*.

Note: Some hosts/devices are not capable of supporting "highest version" in enumeration. Such hosts instead enumerate *all* versions.

For each file that matches the file group *FILES*, the "file commands" in *COMMANDS* are executed in order. Some of the file commands allow aborting the command processing for a given file, effectively filtering the list of files. The interpretation of the different file commands is described below. If *COMMANDS* is NIL, it defaults to (COLLECT), which

collects the matching file names in a list and returns it as the value of DIRECTORY.

The "file commands" in *COMMANDS* are interpreted as follows:

P	Prints the file's name. For readability, DIRECTORY strips the directory from the name, printing it once as a header in front of each set of consecutive files on the same directory.
PP	Prints the file's name without a version number.
a string	Prints the string.
READDATE, WRITEDATE CREATIONDATE, SIZE LENGTH, BYTESIZE PROTECTION, AUTHOR TYPE	Prints the appropriate information returned by GETFILEINFO (see above).
COLLECT	Adds the full name of this file to an accumulating list, which will be returned as the value of DIRECTORY.
COUNTSIZE	Adds the size of this file to an accumulating sum, which will be returned as the value of DIRECTORY.
DELETE	Deletes the file.
DELVER	If this file is not the highest version of files by its name, delete it.
PAUSE	Waits until you type any character before proceeding with the rest of the commands (good for display if you want to ponder).

The following commands are predicates to filter the list. If the predicate is not satisfied, then processing for this file is aborted and no further commands (such as those above) are executed for this file.

Note: if the P and PP commands appear in *COMMANDS* ahead of any of the filtering commands below except PROMPT, they are postponed until after the filters. Thus, assuming the caller has placed the attribute options after the filters as well, no printing occurs for a file that is filtered out. This is principally so that functions like DIR (below) can both request printing and pass arbitrary commands through to DIRECTORY, and have the printing happen in the appropriate place.

PROMPT *MESS*	Prompts with the yes/no question *MESS*; if user responds with No, abort command processing for this file.
OLDERTHAN *N*	Continue command processing if the file hasn't been referenced (read or written) in *N* days. *N* can also be a string naming an explicit date and time since which the file must not have been referenced.
NEWERTHAN *N*	Continue command processing if the file has been written within the last *N* days. *N* can also be a string naming an explicit date and time.

Note that this is not quite the complement of OLDERTHAN, since it ignores the read date.

BY *USER* Continue command processing if the file was last written by the given user, i.e., its AUTHOR attribute matches (case insensitively) *USER*.

@ *X* *X* is either a function of one argument (*FILENAME*), or an arbitrary expression which uses the variable FILENAME freely. If *X* returns NIL, abort command processing for this file.

The following two commands apply not to any particular file, but globally to the manner in which directory information is printed.

OUT *FILE* Directs output to *FILE*.

COLUMNS *N* Attempts to format output in *N* columns (rather than just 1).

DIRECTORY uses the variable DIRCOMMANDS as a spelling list to correct spelling and define abbreviations and synonyms (see Chapter 20). Currently the following abbreviations are recognized:

```
        AU   =>    AUTHOR
         -   =>    PAUSE
  COLLECT?   =>    PROMPT " ? " COLLECT
        DA
    DATE   =>    CREATIONDATE
        TI   =>    WRITEDATE
       DEL   =>    DELETE
      DEL?
  DELETE?   =>    PROMPT " delete? " DELETE
       OLD   =>    OLDERTHAN 90
        PR   =>    PROTECTION
        SI   =>    SIZE
   VERBOSE   =>    AUTHOR CREATIONDATE SIZE
                    READDATE WRITEDATE
```

(**FILDIR** *FILEGROUP*) [Function]

Obsolete synonym of (DIRECTORY *FILEGROUP*).

(**DIR** *FILEGROUP* COM_1 ... COM_N) [NLambda NoSpread Function]

Convenient form of DIRECTORY for use in type-in at the executive. Performs (DIRECTORY '*FILEGROUP* '(P COM_1 ... COM_N)).

(**NDIR** *FILEGROUP* COM_1 ... COM_N) [NLambda NoSpread Function]

Version of DIR that lists the file names in a multi-column format. Also, by default only lists the most recent version of files (unless *FILEGROUP* contains an explicit version).